nudgely↗

The uplift model: what it predicts, how it was built, how far to trust it.

Technical notes for the team and for judges' questions. Every figure comes from the result files in the repo (`data/uplift_summary.json`, `data/uplift_transferable.json`) or the code that produced them.

Jack · Team Bongo Gang · AI-GNITION 2026 · 13 Nov 2026

687,029
customers in a real randomised SMS test
Lenta, public dataset

57%
of extra visits caught by the top 30%
Model 2, mean of 7 test splits

38–85%
range across those 7 splits
random targeting catches 30%

10
inputs, all from a shop's receipts
phone, date and amount only

1 What the model predicts

Most retention tools predict **who might leave** (churn). Nudgely predicts something different: **how much a message changes** a customer's chance of coming back. That difference is called **uplift**:

```
uplift = P(comes back | messaged) − P(comes back | not messaged)
```

Churn risk and uplift disagree exactly where money is wasted. A loyal regular has a high chance of returning either way, so a message changes nothing. A customer who is gone for good has a low chance either way. Only uplift separates both of those from the customers a message actually moves. Real scores for three demo customers:

CUSTOMER	NO MESSAGE	MESSAGED	UPLIFT	TYPE	DECISION
Aisha	28.3%	39.5%	**+11.2 pts**	Persuadable	Message her
Hafiz	34.1%	33.0%	**−1.1 pts**	Sure Thing	Coming anyway: a discount is wasted
Ravi	1.8%	2.6%	+0.7 pts	Lost Cause	Nothing brings him back

The three types come from two cut-offs. **Persuadable** is the top 30% by uplift. Of the rest, **Sure Thing** is the top 30% by chance of coming back with no message. Everyone else is a **Lost Cause**. On the Lenta test set the cut-offs were +0.94 points of uplift and a 10.7% chance of returning anyway. For a real shop they are recalculated inside that shop, so the labels always rank its customers against each other.

2 The data: Lenta

Lenta is a large Russian supermarket chain. It ran a real experiment and published the results for uplift-modelling research. We download it with the scikit-uplift library (`notebooks/fetch_lenta.py`, about 145 MB).

FACT	VALUE
Customers	687,029, with 195 columns each
The experiment	A random **75.1%** got an SMS (515,892); **24.9%** got nothing (171,137)
Outcome	`visit`: did they come back to a store afterwards
Result	**10.26%** came back with no SMS, **11.01%** with one: **+0.75 points**, about +7%
Columns	Mostly purchase behaviour: basket value and its variation, gaps between visits, sales by period, promotion share, food share. Plus past response to SMS and Viber, and demographics (gender, age, children)
Source	uplift-modeling.com/en/latest/api/datasets/fetch_lenta.html

Why the coin flip matters. Because who got the SMS was random, the two groups are alike in every other way. Any difference in who came back was caused by the SMS itself, which is the only thing that lets a model learn the effect of a message rather than which kinds of customers happened to be texted.

Fairness. Gender, age and children are removed before training. Targeting uses behaviour only.

3 Method: two models, then subtract

A single customer is never both messaged and not messaged, so their uplift can't be observed directly. The **two-model** method (also called a T-learner) estimates both halves separately:

1. **Model A** learns, from customers who **got** the SMS only, the chance of coming back.
2. **Model B** learns the same from customers who got **nothing**.
3. For any customer, ask both and subtract: **A − B = uplift**.

SETTING	VALUE
Library	scikit-uplift `TwoModels(method="vanilla")`
Each of the two models	scikit-learn `HistGradientBoostingClassifier`: up to 300 trees, learning rate 0.05, 31 leaves per tree, early stopping on a 10% validation slice
Training data	A 50% random sample of Lenta (343,514 customers), split 70% train / 30% test, stratified so each part has the same mix of SMS and came-back
Test set	103,055 customers the model never saw
Missing values	Left blank. Gradient boosting handles gaps itself, and filling them with 0 would mean something false (for example "perfectly regular" for a customer who simply didn't visit)

Why gradient boosting, not deep learning. It is the standard winner on small tabular data like receipts, trains in seconds, and needs no tuning to be sensible. Neural uplift models (DragonNet, TARNet) need far more outcome data than a small shop produces.

4 Model 1 and Model 2

Both use the same method. They differ only in their inputs, and in how they were tested.

	MODEL 1	MODEL 2 (THE ONE WE QUOTE)
Inputs	190 Lenta columns	**10** columns any shop can work out
Needs	Everything Lenta records about a customer	A sales export: phone, date, amount
Tested on	One train/test split	Seven splits (seeds 1–7), averaged
Top 30% catch (same 7-split test)	44.8% (range 30.6–63.5%)	**57.0%** (range 37.7–84.6%)
Qini AUC (mean)	0.0069	0.0106
Training time, 7 splits	200 s	26 s
Used for	The 30 fictional demo customers' scores	Scoring a real shop (`/shop` , Your sales)
Code / output	`train_uplift.py` → `uplift_summary.json`	`train_transferable.py` → `model_transferable.pkl`

The story in one line: Model 1 was better on paper but useless to a real shop, because no shop has 190 columns about its customers. Model 2 uses only what a till records, and lost no accuracy. Its higher average (57% vs 44.8%) is **within the noise**, so we say "cutting to ten inputs cost nothing", not "the small model wins". On a single split Model 1 scored 52.9% (90% bootstrap interval 31–89%); that is a different measurement and is never quoted beside 57%.

Model 2's ten inputs

Each is calculated per customer from their purchases, as of the last date in the sales file (`nudgely/scoring.py`). "15d", "1m" and "3m" are the last 15 days, 30 days and 91 days.

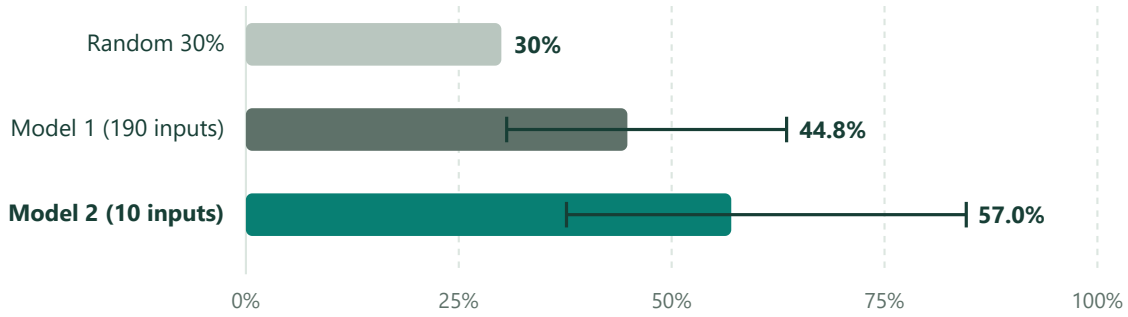
INPUT	IN PLAIN WORDS	HOW IT'S CALCULATED
`months_from_register`	How long they've been a customer	Days since first purchase ÷ 30.4
`k_var_days_between_visits_15d` `…_1m` , `…_3m`	How irregular their visits are, over three windows	Spread of the gaps between visits ÷ their average (coefficient of variation)
`stdev_days_between_visits_15d`	How much the gaps vary, in days	Standard deviation of gaps in the last 15 days
`perdelta_days_between_visits_15_30d`	Visiting more or less often lately	Recent average gap vs the 30-day average gap
`k_var_cheque_15d` , `k_var_cheque_3m`	How much their basket size varies	Coefficient of variation of purchase amounts
`response_sms` , `response_viber`	Replied to a message before	From past campaigns. A new shop has none, so these stay blank

Anything needing two visits in a window is left blank when there's only one. Lenta has the same gaps (77,000 customers have no 15-day variation), so the model learned on data shaped this way. The saved Model 2 is the one trained on the seventh split.

5 How good is it

The main measure: share of extra visits caught by the top 30%

Rank the test customers by predicted uplift and take the top 30%. In that group, compare the return rate of those who got the SMS with those who didn't, and multiply by the group's size: that's how many extra visits the SMS caused there. Divide by the extra visits it caused across everyone. Picking 30% at random catches 30% by definition; a useful model catches more.



Mean of 7 train/test splits; the whiskers show the lowest and highest split. Even Model 2's worst split (37.7%) beat random.

Why the range is so wide. The SMS moved visits by only 0.75 of a point, so the signal is small and the test set's handful of extra visits swings a lot from one split to the next. One split alone moved the figure by 30 points, which is why every headline number is an average over seven, always quoted with its range. The Qini AUC (the area between the model's cumulative gain curve and random) agrees: 0.0106 for Model 2 against 0.0069 for Model 1.

Does the ranking hold across the whole list? (Model 1, single split)



Ten equal groups, 1 = highest predicted uplift. Bars: real extra visit rate, SMS minus no SMS.

From `uplift_summary.json`, 103,055 test customers, about 10,300 per group.

Read this honestly. The top group really does respond most (+2.3 points, three times the average), and groups 1–3 are all above average, which is what "message these first" needs. But the bottom group, predicted least movable, still showed +1.5 points. Groups 1 and 10 both hold Lenta's most frequent shoppers (26% and 20% came back with no SMS, against 10% overall), and the model sorts the movable ones at the top better than the unmovable ones at the bottom. So the ranking is trustworthy for **who to message first**, not for exact per-person numbers or for writing anyone off.

6 How the product uses it

STEP	WHAT HAPPENS	WHERE
1. Read the sales file	CSV or Excel. Columns found by name ("Mobile No", "Date", "Total (SGD)"). Phones cleaned so `+65 9123 4567` and `9123-4567` match; dates read day-first; bad rows skipped and counted	`nudgely/shop_scores.py`
2. Ten numbers per customer	The inputs in section 4, as of the last date in the file	`nudgely/scoring.py`
3. Score	Model 2 gives each customer a chance with and without a message, the uplift, and a label relative to the shop	`score_sales()`
4. Plan	Top Persuadables by uplift; a random share kept back (default 10%). Expected extra visits = the sum of the messaged customers' uplift, an estimate	`nudgely/shops.py`
5. Message	The AI writes one checked sentence into an approved WhatsApp template. **The model plays no part in the wording**: it decides who, not what	`write_messages()`
6. Measure	A later sales file marks who bought within 30 days, messaged and kept back alike. Result = the difference in return rates, with a 90% range (Newcombe's method on Wilson intervals, sound on small groups) and a verdict: worked, didn't help, or too early to call	`nudgely/returns.py`

7 Limitations: say them before a judge does

- **The evidence is overseas.** A Russian supermarket, by SMS, in 2020. Not a Singapore neighbourhood shop on WhatsApp. A local pilot has to prove it.
- **No recency input.** Lenta has no "days since last purchase", so the model can't use the most obvious drift signal. Its windows partly stand in for it.
- **Small, noisy signal.** The SMS added 0.75 of a point. Hence the 38–85% range and the honest reading of the decile chart.
- **Labels are relative.** "Persuadable" is the top 30% of this shop, not an absolute promise that a message will work.
- **Blank inputs for new shops.** "Replied before" is unknown until a shop has run campaigns.
- **WhatsApp chats aren't model inputs yet.** The AI tags them for writing messages, but nothing turns them into numbers the model learns from, and chats aren't yet linked to sales customers.
- **Demo customers use Model 1.** The 30 fictional Kampung Fresh customers were scored before Model 2 existed.

8 What comes next

STEP	WHAT	NEEDS
1	Fill "replied before" from WhatsApp replies	Link chats to sales customers by matching scrambled phone numbers
2	Retrain on the shop's own campaigns, still with gradient boosting, blending Lenta with the shop's rows	A few hundred customers with known outcomes, from campaigns that keep a group back
3	Keep a random kept-back group in every campaign, forever	Nothing: it's built in. Training only on customers the model chose to message would slowly bias it
4	Neural uplift models (DragonNet, TARNet)	Tens of thousands of real outcomes, and a plateau from step 2

9 Reproduce it

COMMAND	PRODUCES
`python notebooks/fetch_lenta.py`	Downloads Lenta to `data/raw/lenta.parquet` (git-ignored)
`python notebooks/train_uplift.py`	Model 1, decile table, single-split numbers → `data/uplift_summary.json`
`python notebooks/check_capture.py`	The 52.9% and its 31–89% bootstrap interval
`python notebooks/train_transferable.py`	Both models over 7 splits → `data/uplift_transferable.json`, and Model 2 → `data/model_transferable.pkl`
`python scripts/score_shop.py --sales file.csv`	Scores any sales export from the command line

Dataset: uplift-modeling.com/en/latest/api/datasets/fetch_lenta.html · Library: scikit-uplift 0.5.1, scikit-learn 1.9.1 · Plain-English version: `docs/HOW_THE_MODEL_WORKS.md` · Numbers policy: `docs/Numbers.md`